



GENOME



MUTATION



AMR



QC



SAMPLES



VARIANTS



SCIENCE LAB



RADIATION



RESEARCH



FORGE



Design thesis

The interface should feel like a serious scientific workstation, not a sci-fi toy.

1. FAMILIAR DESKTOP BEHAVIOR

Users should instantly understand double-click, drag, resize, minimize, maximize, taskbar, Alt+Tab-style switching, context menus and scrollbars.

2. 2000S ATMOSPHERE, MODERN EXECUTION

Use dense chrome, beveled separators, dark graphite surfaces, restrained green/cyan status accents and tiny system labels. Avoid noisy neon, excessive glassmorphism and literal film replicas.

3. DATA IS THE HERO

Scientific tables, sequence views, heatmaps and evidence panels should dominate app windows. Decoration sits behind the data.

4. EVERY ACTION LEAVES CONTEXT

Status bars, provenance chips, selected sample IDs, model version, compute state and timestamps should be visible but quiet.

5. WINDOWED, NOT ROUTE-BASED

Every application opens as a desktop window. Do not navigate away from the desktop for a major workflow.

Visual language

Original identity inspired by early-2000s biotech/corporate thriller software.

BACKGROUND

PANEL

GRID

TEXT

GREEN

CYAN

AMBER

DANGER

The visual target is **industrial precision**: thin rules, compact labels, squared data modules, hard edges softened only enough for desktop windows, and color used to encode state instead of mood.

SURFACE RULES

- Base radius: 5-7px. Avoid rounded “mobile card” styling.
- Window chrome: 24px title bar, 1px stroke, 4px internal padding.
- Primary text: near-white. Secondary: desaturated blue-green.
- Green = active/healthy/success. Cyan = information/computation. Amber = warning/needs review. Red = failure/destructive action.
- Use subtle noise/grain only in wallpaper, never inside data tables.



GENOME



MUTATION



AMR



QC



SAMPLES



VARIANTS



SCIENCE LAB



RADIATION



RESEARCH

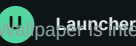


FORGE

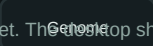
1 APP DESKTOP



2 TASKBAR / WINDOW SWITCHER



Launcher



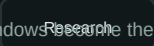
Genome



Mutation



AMR



Research

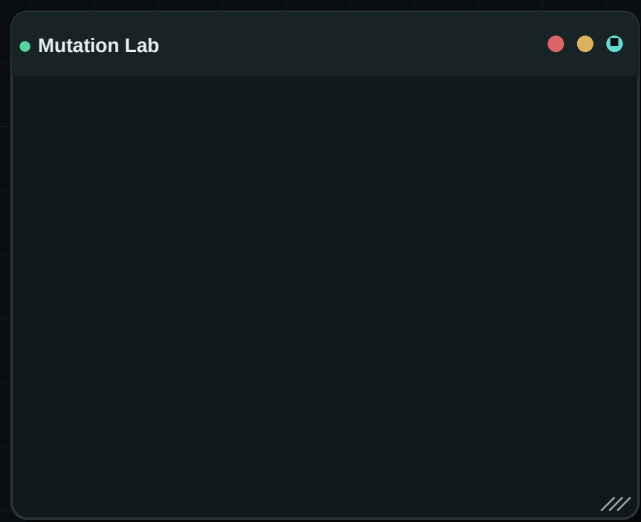
WSL 1 2 JOBS 1 SECURE

The desktop is intentionally quiet. The app desktop should have enough negative space that windows don't compete for visual hierarchy. App icons use a single 42x46px tile with acronym + full label. Double-click is the primary launch gesture.

05

Window manager

Exact behavior for double-click, focus, drag, resize and state management.



WINDOW STATE MACHINE

```
CLOSED -> OPEN -> ACTIVE -> INACTIVE
ACTIVE: highest z-index, keyboard target, bright border
MINIMIZED: taskbar only, content remains mounted
MAXIMIZED: snaps to work area, preserves restore bounds
CLOSE: destroy window state + release subscriptions
OPEN SAME APP: create a new instance only if app declares multi-instance=true
```

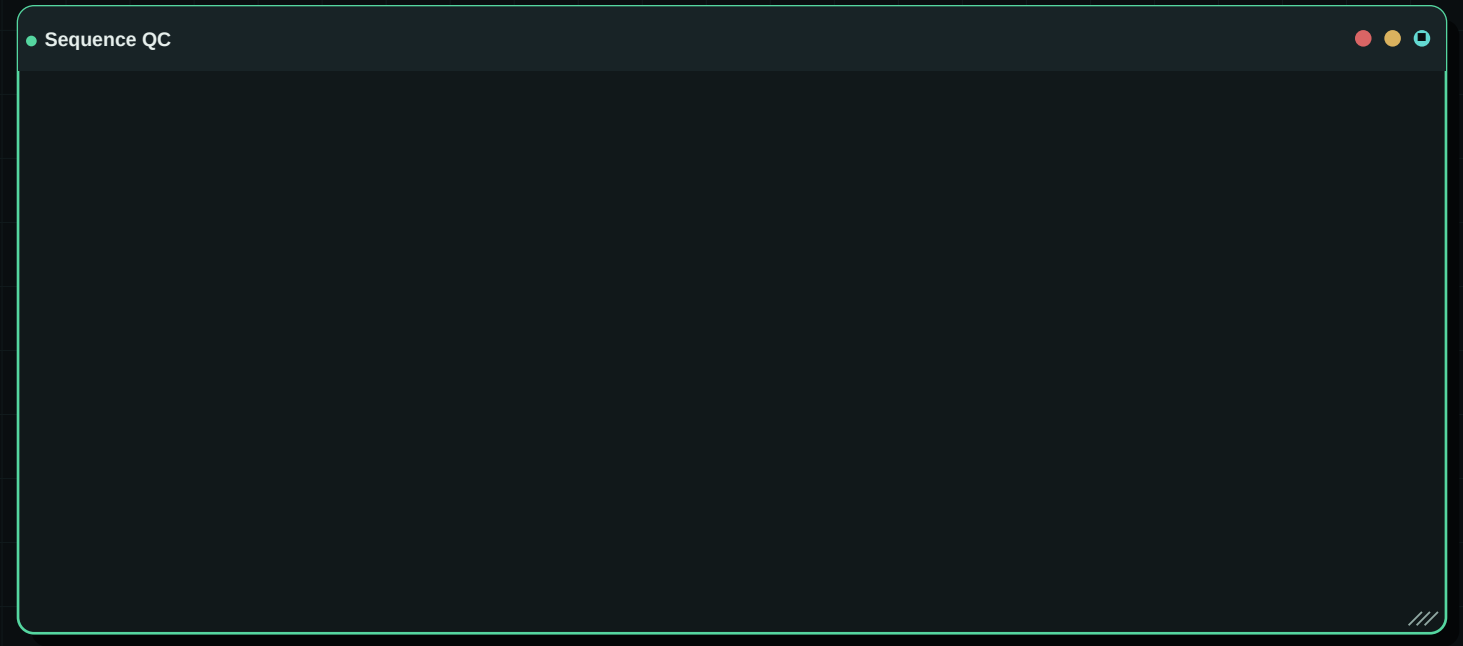
Double-click: icon opens a new app window. **Single click:** selects icon. **Click title bar:** focuses window. **Drag title bar:** moves window. **Drag edges/corners:** resizes. **Double-click title bar:** toggles maximize. **Esc:** closes transient dialogs, not the app.

06

Window chrome

Keep the title bar dense, technical and predictable.

- X Close
- Minimize
- Max / restore

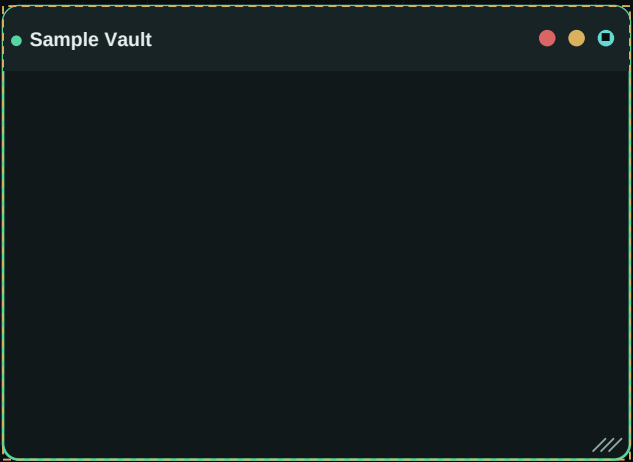


Area	Behavior
Left	App status dot + application name
Center	Optional document/sample name; never use this for navigation
Right	Close / minimize / maximize
Double-click	Maximize/restore
Right-click	Window context menu
Status strip	Optional: job state, sample ID, model version, timestamps

07

Drag + resize rules

Window manipulation must work with mouse and keyboard without accidental content clicks.



Resize zone: 8px transparent hit area around every edge + corner

RECOMMENDED CONSTRAINTS

- Global minimum: 420 x 280
- Data-app minimum: 760 x 500
- Report-app minimum: 820 x 560
- Maximized excludes taskbar + system bar
- Keep at least 20% of window visible
- Snap when near work-area edges
- Persist size/position per app

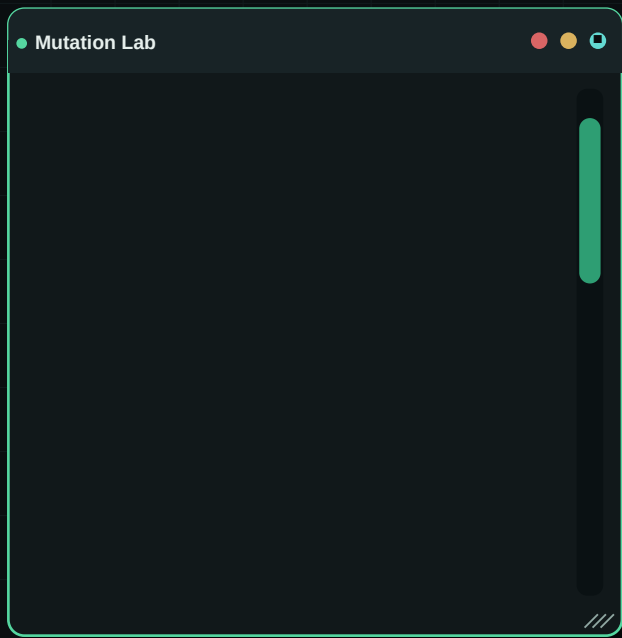
KEYBOARD CONTRACT

Alt+Tab	cycle active windows
Alt+F4	close focused window
Win+M	minimize all non-pinned windows
Win+Shift+F	focus Vella / command bar
Arrow keys	navigate launcher grid
Enter	open selected app
Esc	close tooltip/dialog

08

Scrolling + overflow

Every new window must have its own reliable scrolling context.



1. WINDOW BODY

The body owns vertical scrolling. Title bar and optional app toolbar stay fixed. Never let the entire browser page scroll when a child app is open.

2. SCROLLBAR

Use a 6-10px scrollbar track. Thumb color follows the app accent but remains low-contrast at rest. Thumb expands on hover.

3. INTERNAL PANELS

Long tables can have an internal horizontal scroll; code/log panels may have both axes. Do not stack more than two independent scroll containers unless necessary.

SCROLL RULES

- Wheel scrolls the container beneath the pointer.
- Shift+wheel = horizontal scroll where supported.
- PageUp/PageDown = app-body scroll.
- Home/End = beginning/end of focused scroll region.
- Show "N rows" + sticky table headers for long datasets.
- Virtualize rows when list > 1,000 items.

Hover text + tooltips

Tooltips are mandatory for icons and should feel like native system chrome.

TOOLTIP SYSTEM



Delay 450-650ms. If the pointer moves away, hide immediately. Tooltip should include label + optional shortcut + one-line explanation. Never use tooltips to reveal critical information that is otherwise inaccessible.

TOOLTIP CONTENT RULES

- Icon-only buttons: always tooltip.
- Destructive action: label + consequence.
- Data badge: definition + source.
- Model score: name + meaning, not only “92%”.
- Disabled action: tooltip must explain why.

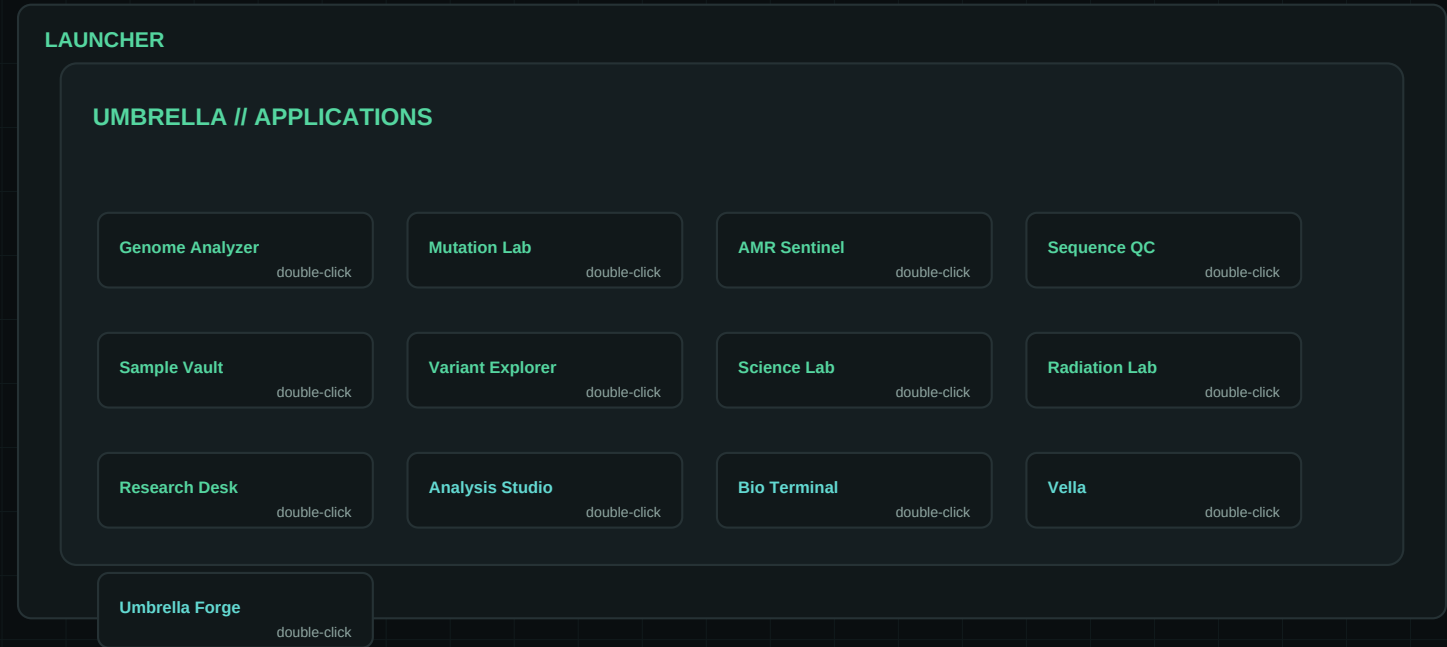
HOVER STATES

rest	100% base fill
hover	+1 surface level, accent edge
active	accent line + pressed offset
focus	2px external outline
disabled	45% opacity, tooltip explains

10

App launcher

The launcher should feel like a 2000s workstation start menu, not a mobile app grid.



Search field is persistent at the top. Typing filters the launcher instantly. Each result supports single-click selection, Enter to open, and double-click to open directly.

11

Typography + density

The interface should look information-rich without becoming visually noisy.

UMBRELLA

SYSTEM STATUS / MODULE

Body copy, table values and explanations use a clean neutral sans.

GENOME 5.21 Mb // QC PASS // BUILD 0.9.4

TYPE RULES

- Display 28-36px only for identity.
- Section 16-20px.
- App title 7-9px semibold.
- Body 8-10px, 1.35 line-height.
- Metadata 6-7px uppercase.
- Monospace for IDs, sequences, logs and raw model output.
- All-caps only for labels, not paragraphs.

TABLE DENSITY

```
header  24px row, sticky, accent underline
body    22px row minimum; 7-8px text
hover   entire row highlight; no row-shifting animations
selected left accent stripe + subtle background
long lists  virtualized after ~1,000 rows
```

Core component library

Build the desktop shell from primitives before building any scientific app.

Window

TitleBar

Taskbar

AppIcon

Launcher

Tooltip

ContextMenu

Modal

Tabs

DataTable

Timeline

SequenceViewer

MetricCard

StatusChip

ProgressBar

JobConsole

CommandPalette

Toast

PermissionDialog

EmptyState

COMPONENT CONTRACT

Every component gets keyboard focus behavior, hover state, disabled state, data-loading state and error state from day one. Scientific apps should not invent their own button, modal, table or tooltip implementations. Use the same shell primitives everywhere.

```

ui/
desktop/ Window.tsx Taskbar.tsx Launcher.tsx
chrome/ TitleBar.tsx Tooltip.tsx ContextMenu.tsx
data/ DataTable.tsx Timeline.tsx SequenceViewer.tsx
feedback/ Toast.tsx JobConsole.tsx ProgressBar.tsx
security/ PermissionDialog.tsx StatusChip.tsx
    
```

Application contract

Every native app is a windowed module with a shared lifecycle and manifest.

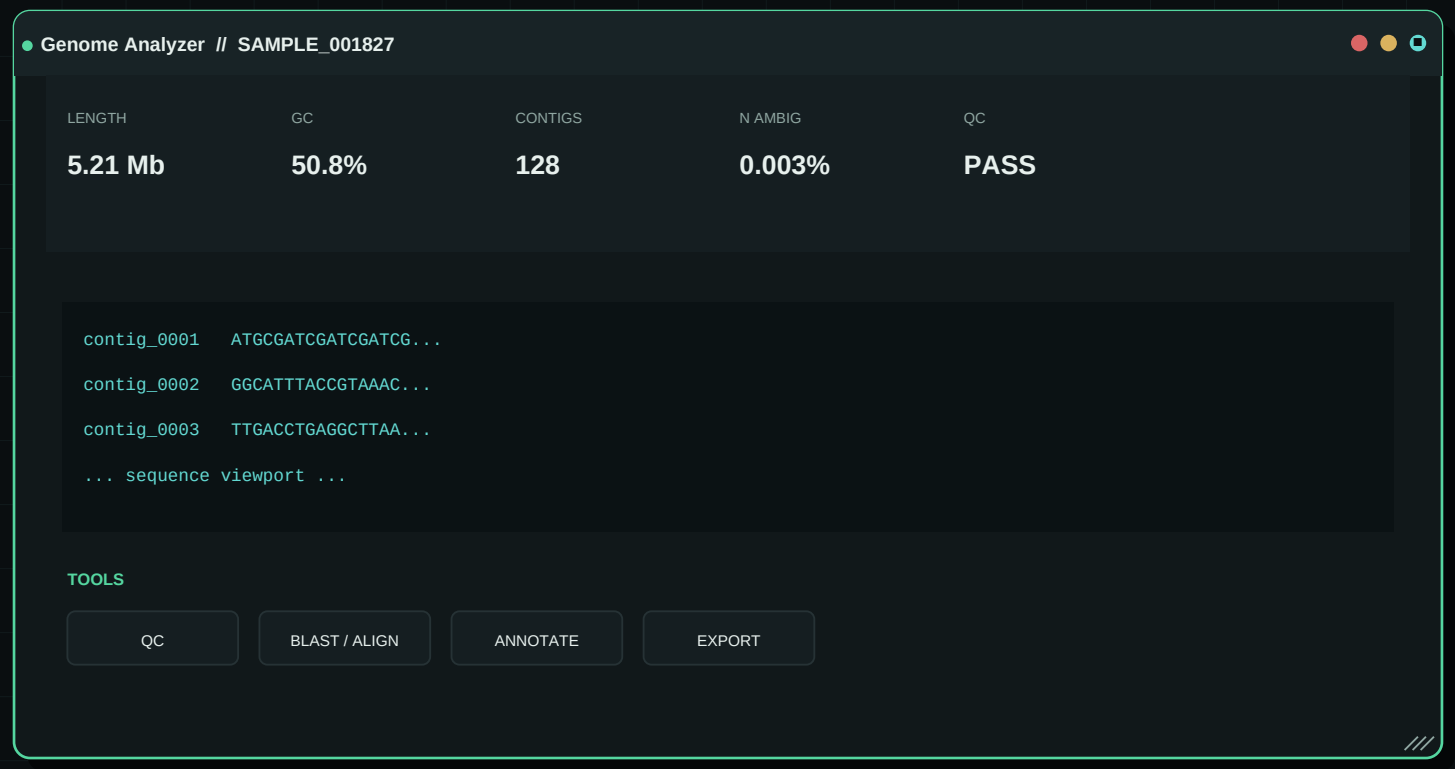
```
export type UmbrellaApp = {
  id: string; name: string; icon: string;
  minSize: { w: number; h: number };
  multiInstance?: boolean;
  permissions: Permission[];
  open(ctx: AppContext): void;
  render(): JSX.Element;
  destroy(): void;
};
```

Key	Purpose
id	Stable launcher/runtime identifier
permissions	Explicit capability requests
minSize	Window manager constraint
multiInstance	Whether double-click creates multiple instances
menu	Commands shown in app context menu
telemetry	Events and performance counters enabled by default

14

Genome Analyzer

Primary sequence inspection window.



UX goal: a scientist can load a sample, inspect summary statistics, jump into a sequence region, run a tool and keep the current sample context without opening a new browser page.

15

Mutation Lab

Reference vs sample comparison with a persistent evidence pane.

Mutation Lab // REF_A vs SAMPLE_1827

REFERENCE

SAMPLE

5' ATGCGATCGATCGATCGATC... 3'

5' ATGCGATGGATCGATCGATC... 3'

POS	TYPE	REF	ALT	EVIDENCE
10231	SNP	C	T	HIGH
18442	SNP	A	G	MED
21980	DEL	GAC	-	HIGH
...	...	...	...	...

VARIANT DETAIL

Selected variant
chr/contig: contig_12
position: 10,231
change: C → T
evidence: high

Show provenance, model output
and supporting annotations here.

///

Use a sticky reference/sample header and never make the scientist remember which sequence is selected. All visual highlights should remain visible while scrolling.

AMR Sentinel

The operational successor to the original Genome Firewall engine.

● AMR Sentinel // SAMPLE_1827

RESISTANCE SIGNAL

ELEVATED

evidence-backed output / model v0.9

Drug class	Signal	Evidence
Carbapenem	HIGH	Marker
Tetracycline	HIGH	Marker
Macrolide	MED	Marker
Fluoroquinolone	MED	Mutation

DETECTED MARKERS

- blaNDM
- blaOXA
- tetM
- ermB
- gyrA

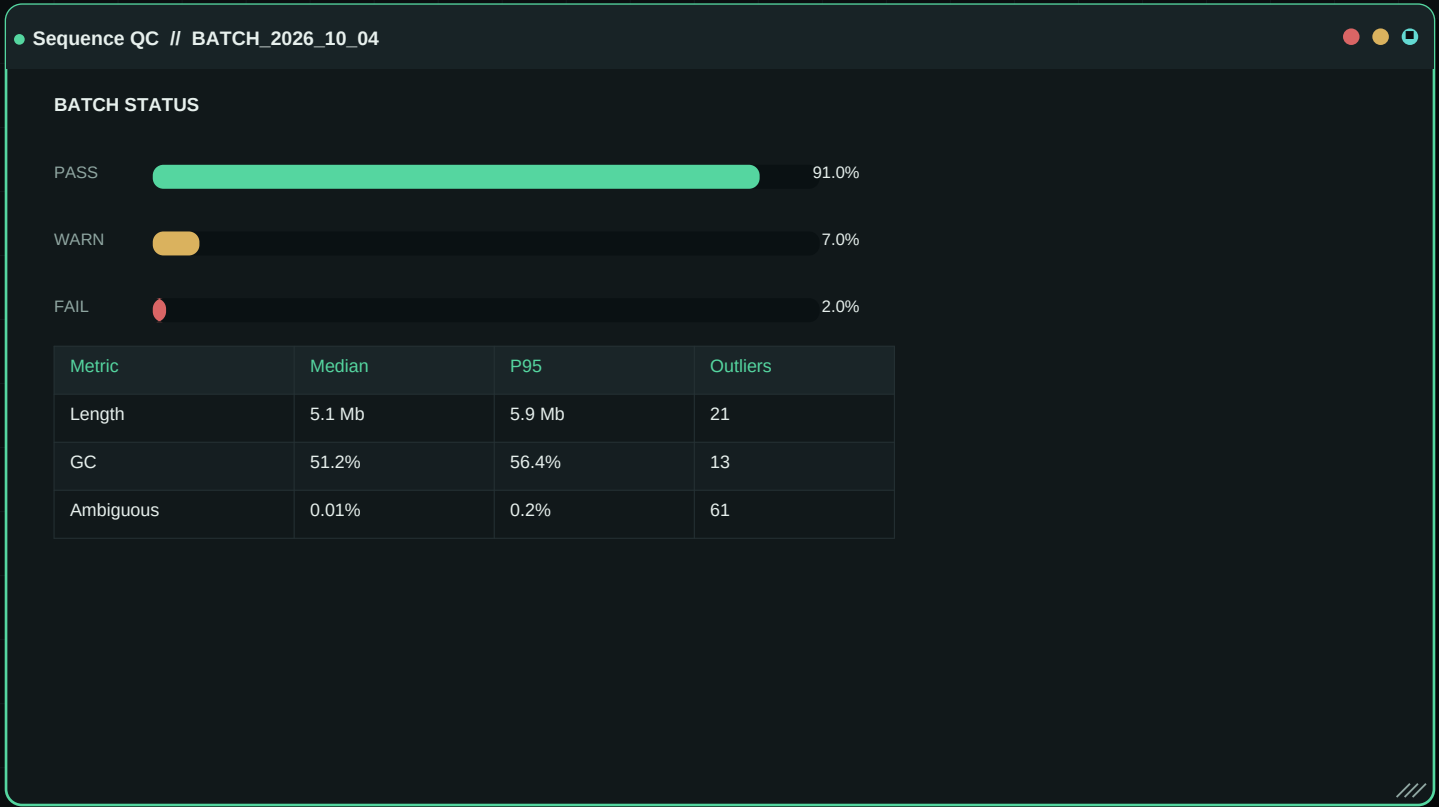
///

Never turn a scientific evidence chain into one mysterious “trust score”. Show evidence class, reference source/version, sample metadata and model confidence separately.

17

Sequence QC

Batch-quality control window built for thousands of records.



Use virtualization and background jobs. The UI should remain responsive while the backend parses and computes. Show a live job console in the bottom 20% of the app window.

18

Sample Vault

The persistent identity layer tying sequences, experiments and reports together.

● Sa

ID	ORGANISM	SOURCE	QC	AMR	UPDATED
1827	E. coli	BV-BRC	PASS	HIGH	22:41
1828	Klebsiella	BV-BRC	WARN	MED	22:39
1829	Staph.	Local	PASS	LOW	22:35
1830	Candida	Public	PASS	N/A	22:31

SAMPLE SNAPSHOT

SAMPLE_1827

Genome 5.21 Mb

QC PASS

5 AMR markers

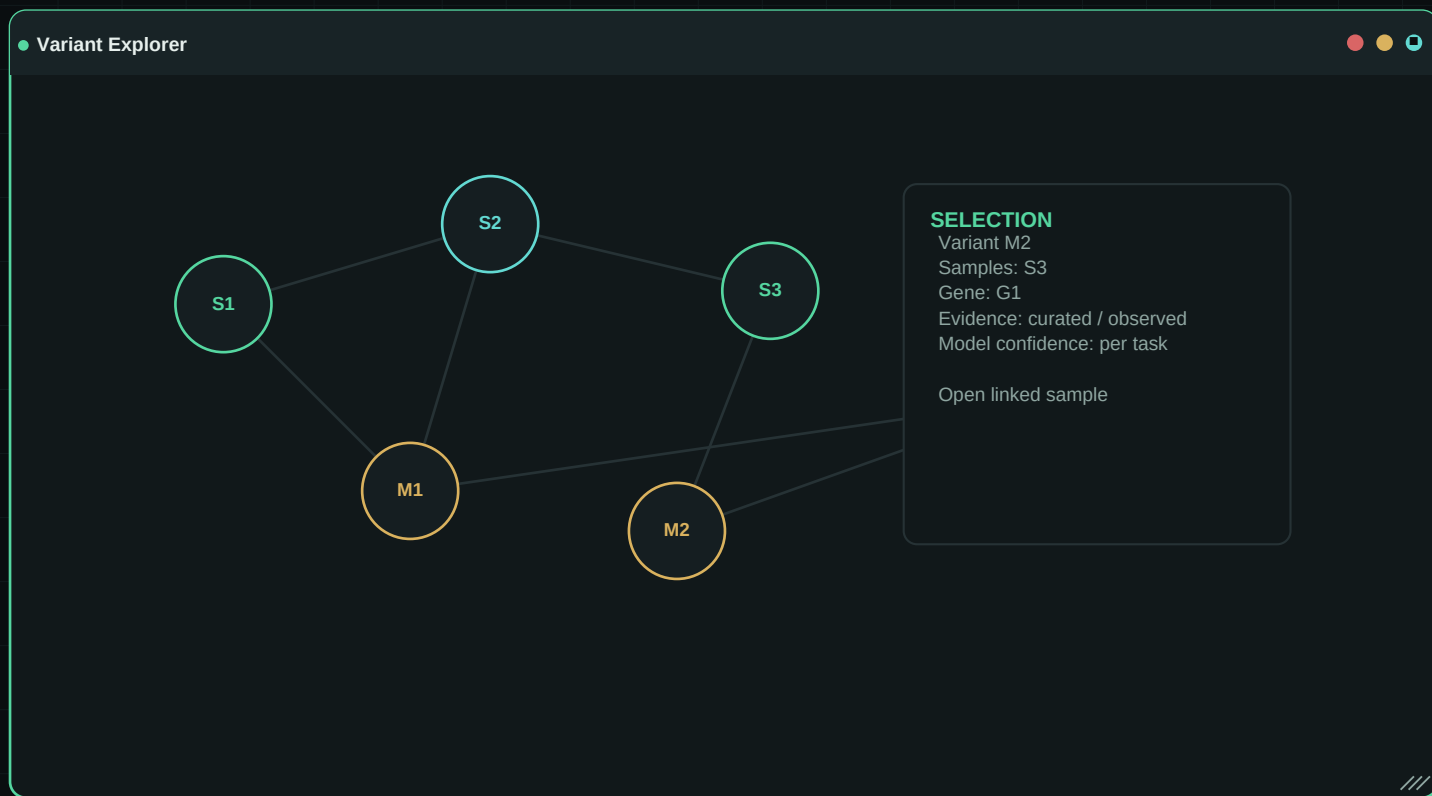
3 linked reports

2 analyses queued

Double-click a row to open the sample details window. Row click selects without opening. Ctrl/Cmd-click supports multi-select.

Variant Explorer

Graph-like navigation without forcing a graph into every screen.

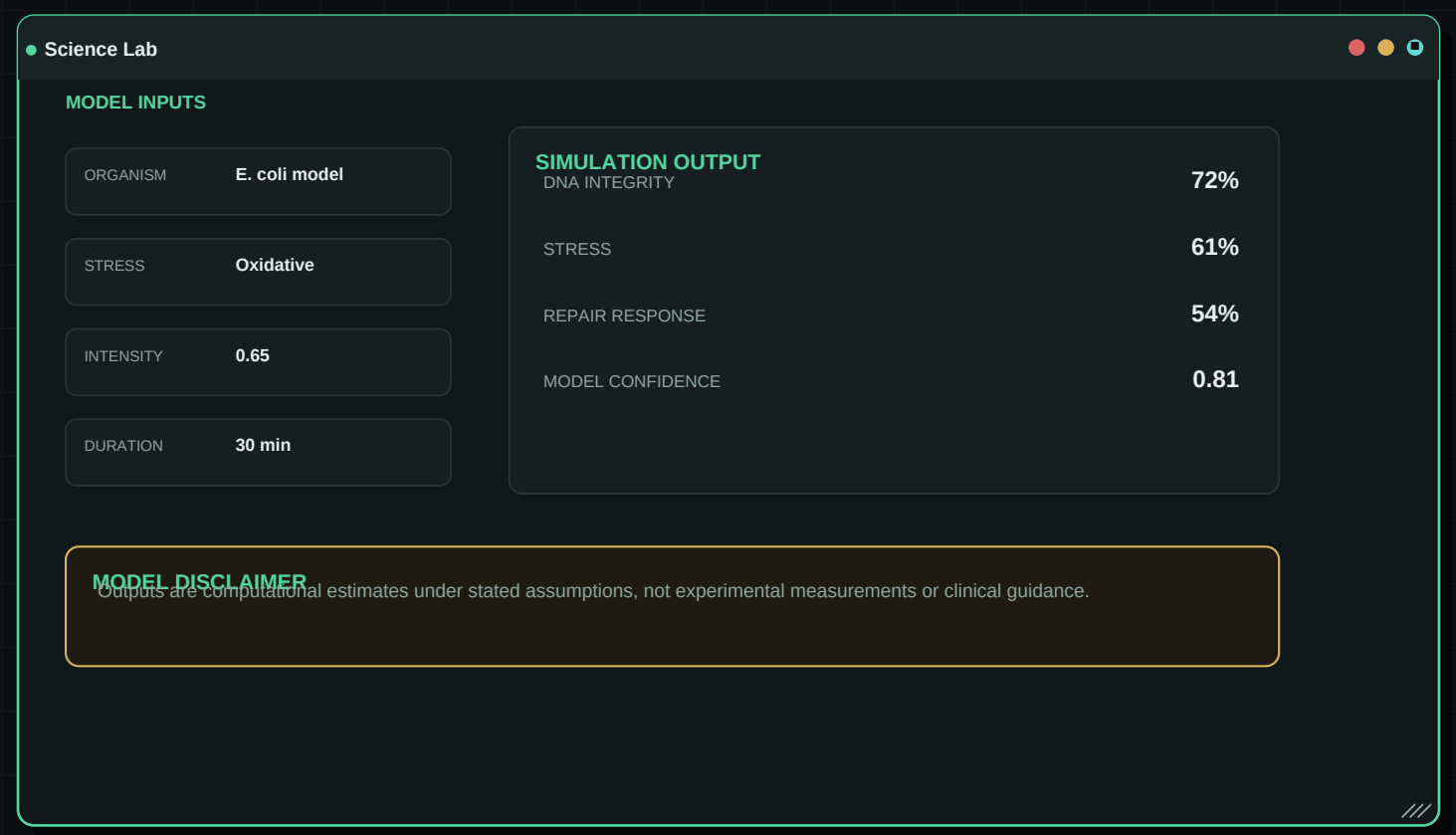


Use the graph only for relationships. Put exact values and evidence in adjacent structured panels. Clicking a node should not destroy the current navigation context.

20

Science Lab

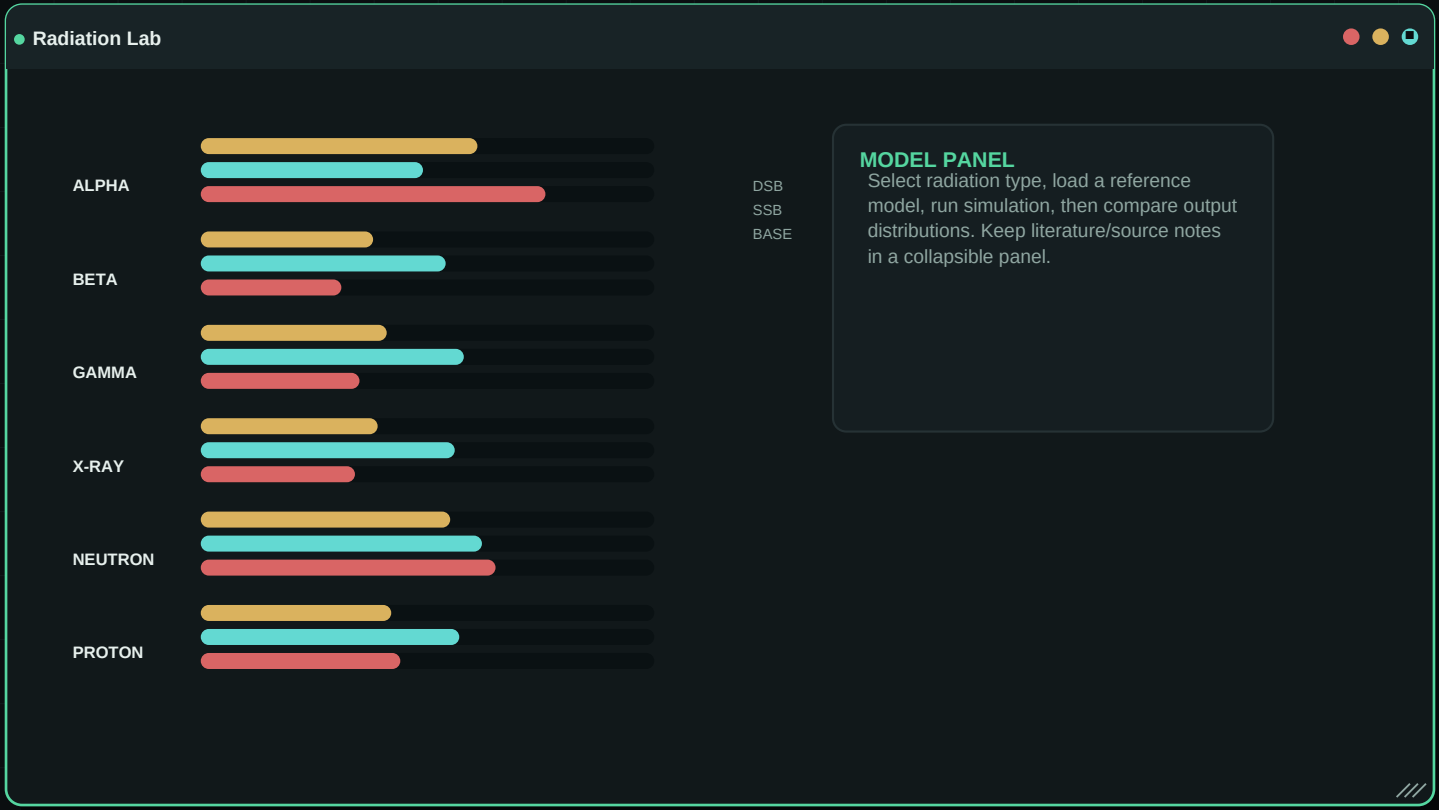
A safe computational simulation workspace for hypothesis exploration.



This app is where you can make the product visually impressive without pretending the simulation is a validated wet-lab predictor.

21 Radiation Lab

Compare modeled DNA-damage patterns by radiation category.

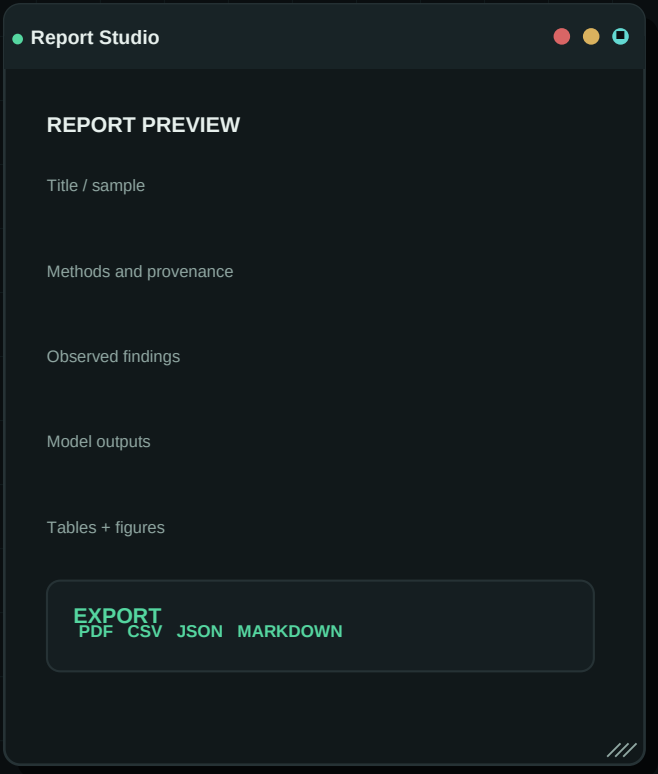
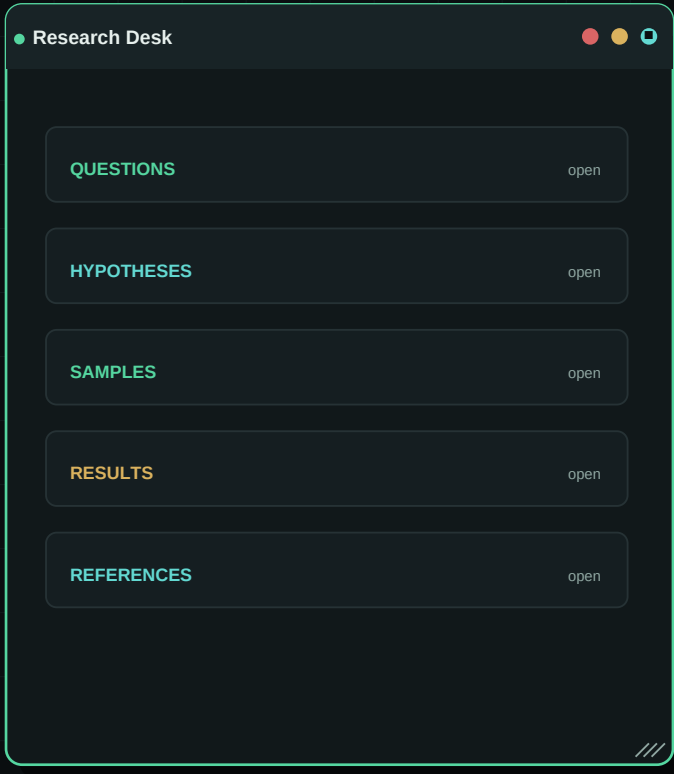


UI requirement: never imply a “most dangerous radiation” answer without context such as dose, energy, geometry, biological system and model assumptions.

22

Research Desk + Report Studio

Persistent scientific workspace and reporting surface.

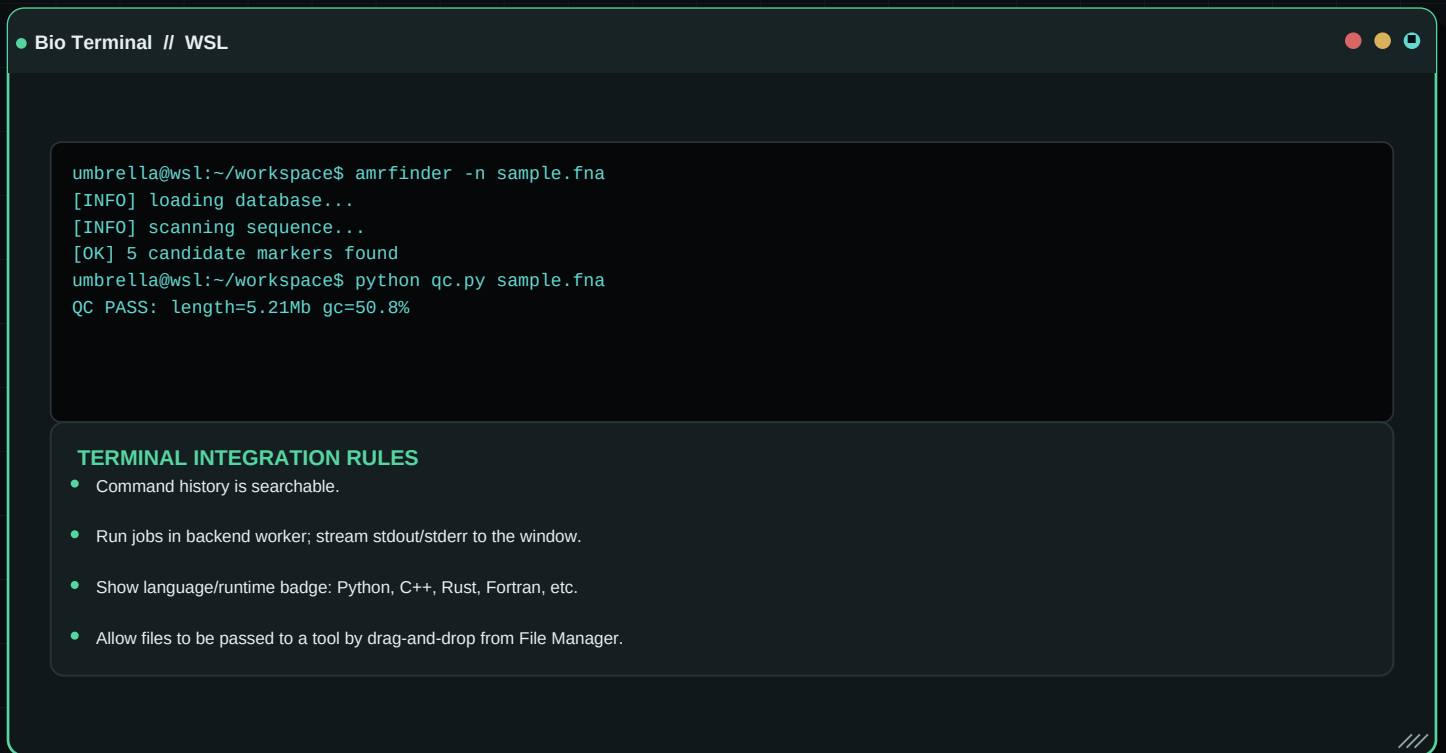


Research Desk should preserve context across app windows. Report Studio should be able to consume selected sample(s), selected variant(s) or a completed job directly from the taskbar/context menu.

23

Bio Terminal

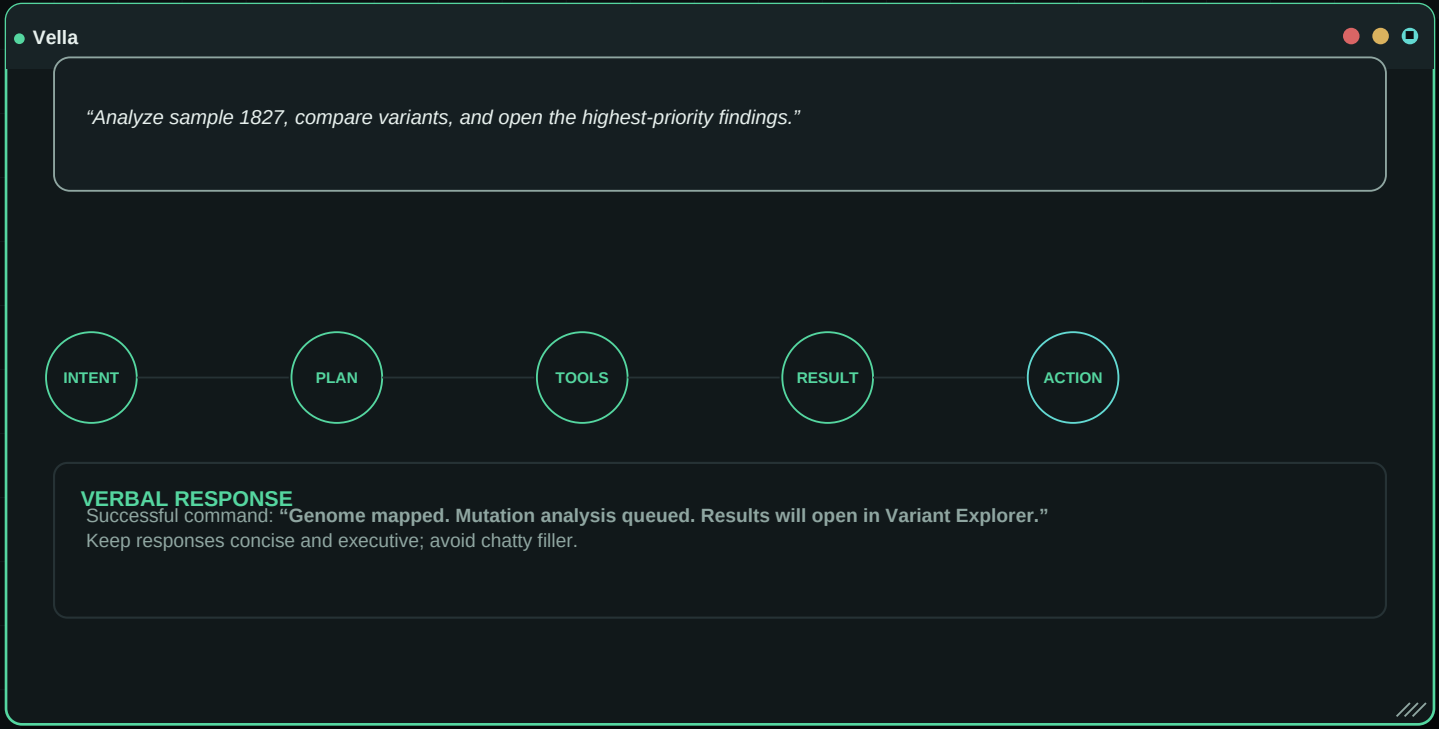
Native-looking terminal integration for WSL and scientific command-line tools.



Do not fake shell output. The terminal is a real entry point to the scientific backend, with explicit sandbox/permission boundaries.

Vella

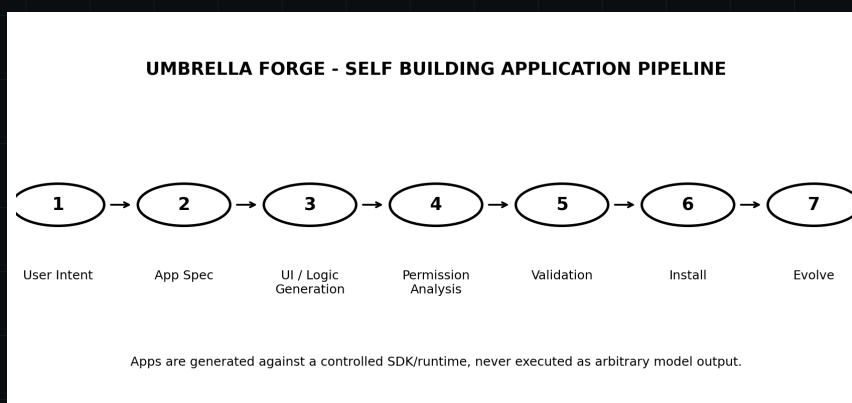
The scientific control layer: voice/text intent -> tools -> windows -> results.



UI requirement: Vella must be able to focus an existing window, open a new one, pass selected sample IDs, and surface a job result as a taskbar notification.

Umbrella Forge

The self-building app surface: request -> spec -> build -> validate -> install.



APP GENERATION PIPELINE

```
User intent
-> Intent extraction
-> App spec JSON
-> Component tree / actions
-> Permission review
-> Sandbox build + tests
-> Install into App Registry
-> Window opens as first-class desktop app
```

Every generated app must use Umbrella primitives: Window, Toolbar, DataTable, Forms, Charts, SequenceViewer and Notifications. Generated code must never bypass the capability model.

Self-built app UX

How a generated application should feel after it is installed.

Custom app // Sequence Diff

GENERATED APP

REFERENCE	SAMPLE
ATGCGATC...	ATGCGATG...
	^

GENERATION RECEIPT

Spec: passed

OK

Permissions: 1 requested

OK

Tests: 14 passed

OK

Runtime: native

OK

Created: 22:42

Version: 1.0.0

EVOLUTION

When a user asks “**add export + a GC-content chart**”, Forge creates a patch specification against the installed app, validates it, increments the version and reloads the same window. Never destroy and recreate the user’s entire workspace.

```
App v1.0 -> modification spec -> validator -> v1.1 -> hot reload
Preserve window position, scroll state and selected sample when safe.
```

27

Frontend implementation stack

Recommended stack for the desktop shell and scientific apps.

Next.js 15	App shell, routing for auth/settings only, server actions where useful
React 19	Component/runtime layer
TypeScript	Strict types for app manifests, window state, scientific schemas
Tailwind CSS	Tokenized surfaces and utility composition
Framer Motion	Only for restrained window transitions / hover polish
Zustand	Window manager, taskbar, app registry, global session state
React Aria / Radix	Accessible primitives where custom primitives are expensive
TanStack Table	Virtualized scientific tables
Monaco Editor	Bio Terminal / generated-code inspection
xterm.js	Real WSL terminal window
Web Workers	Client-side lightweight parsing/formatting only
WebSocket / SSE	Streaming job state, logs and model output

28

Window + state architecture

Separate desktop state, app state and server job state.

```
/src
/desktop      WindowManager, Desktop, Taskbar, Launcher
/apps         Genome, Mutation, AMR, QC, Samples, Lab...
/runtime      AppRegistry, ManifestValidator, CapabilityGate
/vella        Planner, ToolRouter, VoiceState
/forge        SpecParser, Builder, Validator, Installer
/jobs         JobStore, SSE client, JobConsole
/lib          domain types, schemas, formatting
```

State	Owner	Examples
Desktop	Zustand	windows, z-order, focus, taskbar
App local	React/Zustand	selected sample, tabs, filters
Server job	DB + stream	job progress, stdout, artifacts
Persisted	DB	window geometry, app installs, preferences
Scientific	Backend	results, model version, provenance

The browser URL should represent only truly navigational states (auth, settings, share links). App windows are local desktop state, not routes.

29

Implementation phases

Build order for the Kiro sprint, with polish gates between phases.

PHASE 1 - SHELL	Desktop + taskbar + launcher + WindowManager	2.0h
PHASE 2 - CHROME	Drag/resize/maximize/minimize + tooltips + context menu + scroll conta	2.0h
PHASE 3 - RUNTIME	App manifest + registry + permissions + app loading	2.0h
PHASE 4 - CORE APPS	Genome Analyzer + Mutation Lab + AMR Sentinel	3.0h
PHASE 5 - DATA WINDOWS	Sample Vault + QC + Variant Explorer	2.0h
PHASE 6 - VELLA	Command bar, tool routing, window focus/open actions	2.0h
PHASE 7 - FORGE	One generated app end-to-end + validation + install	3.0h
PHASE 8 - DEMO POLISH	Wallpaper, sounds optional, hover states, error states, QA	1.5h

If time collapses, cut secondary apps before cutting the window manager or Forge. The core differentiator is “scientific desktop + self-building app runtime.”

30

Frontend QA checklist

Do not ship the demo until every interaction below works.

- ☐ Double-click icon opens correct window.
- ☐ Single click selects without opening.
- ☐ Window focuses when clicked anywhere in chrome/cont
- ☐ Drag title bar works at every zoom level.
- ☐ Resize works from all edges/corners; no accidental
- ☐ Double-click title bar maximizes/restores.
- ☐ Minimize removes window from desktop but keeps task
- ☐ Close releases subscriptions and job streams.
- ☐ Window remains at least 20% visible when dragged.
- ☐ Each new window has its own scroll context.
- ☐ Long tables have sticky headers and virtualization
- ☐ Every icon-only control has hover text.
- ☐ Tooltip never clips at screen edges.
- ☐ Keyboard focus is visible.
- ☐ Escape closes transient dialogs/tooltips.
- ☐ Generated app requests permissions before access.
- ☐ App error state is readable and recoverable.
- ☐ Loading state never blocks the whole desktop.
- ☐ No page-level scroll while desktop is active.
- ☐ Render at 1366x768 and 1920x1080 without clipped ap

FINAL DESIGN RULE

Umbrella OS should look like software that runs a laboratory, not a web page pretending to be an operating system. The window manager, scientific data density, provenance cues and self-building app capability are the product. Visual effects exist only to reinforce that story.

DOUBLE-CLICK -> NEW WINDOW -> WORK -> SCROLL -> DRAG -> RESIZE -> CLOSE

Every app behaves like a first-class desktop program.